

System Architecture Documentation

1. Project Overview

تم تطوير المشروع باستخدام:
Laravel

كنظام E-Commerce API يدعم:

- إدارة المنتجات
- السلة الشرائية
- الطلبات
- الدفع Checkout
- التقارير
- المعالجة الخلفية
- توزيع الأحمال
- الحماية من الـ Race Conditions

يعتمد النظام على معمارية متعددة الطبقات (Layered Architecture) لفصل المسؤوليات وتحسين قابلية التوسع والصيانة.

2. High-Level Architecture

يتكون النظام من الطبقات التالية:

Layer	Responsibility
Routes Layer	استقبال الطلبات وتوجيهها
Controllers Layer	تنفيذ منطق الـ API
Services Layer	العمليات التجارية الثقيلة
Middleware Layer	مراقبة الأداء والتحكم بالتزامن
Jobs & Queues Layer	تنفيذ العمليات غير المتزامنة
Models Layer	التعامل مع قاعدة البيانات
Database Layer	تخزين البيانات
Load Balancer Layer	توزيع الطلبات بين الخوادم

3. Main Components

3.1 Controllers

تستقبل الطلبات القادمة من المستخدمين وتنفذ العمليات المناسبة.

الملفات الأساسية:

- AuthController
 - ProductController
 - CartController
 - OrderController
 - SafeOrderController
 - DailySalesReportController
 - SystemStatusController
-

3.2 Models

تمثل جداول قاعدة البيانات باستخدام Eloquent ORM.

النماذج الأساسية:

- User
- Product
- Cart
- CartItem
- Order
- OrderItem
- DailySalesReport

3.3 Services

تم استخدام طبقة Services لفصل منطق الأعمال المعقد عن الـ Controllers

DailySalesReportService

المسؤول عن:

- تحليل المبيعات
- معالجة البيانات
- حساب الإحصائيات
- تنفيذ batch processing

3.4 Jobs & Queues

تم استخدام نظام:

Asynchronous Processing

لنقل العمليات الثقيلة خارج مسار الطلب الرئيسي.

الوظائف المستخدمة:

- GenerateInvoiceJob
- SendOrderNotificationJob
- ProcessDailySalesReportJob

يتم تنفيذها باستخدام:

php artisan queue:work

4. Requirement 1 — Data Integrity & Race Condition Protection

المشكلة

عند تنفيذ عدة عمليات Checkout بنفس الوقت، قد يقوم أكثر من مستخدم بشراء نفس المنتج، مما يؤدي إلى:

- Stock inconsistency
- Negative stock
- Duplicate updates

وهذه تسمى:

Race Condition

الحل المستخدم

تم استخدام:
Redis

لتطبيق:

Distributed Locking

داخل:

SafeOrderController

كيف يعمل النظام

قبل تعديل كمية المنتج:

```
Cache::lock('product_'.$item->product_id, 10)
```

يتم:

1. إنشاء Lock على المنتج
2. منع أي Request آخر من تعديله
3. تنفيذ العملية بأمان
4. تحرير الـ Lock بعد الانتهاء

5. Requirement 2 — Resource Management & Capacity Control

الهدف

منع النظام من استهلاك الموارد بشكل مفرط عند زيادة عدد الطلبات المتزامنة.

الحل المستخدم

تم إنشاء Middleware باسم:

```
ConcurrencyLimitMiddleware
```

آلية العمل

يقوم الـ Middleware بـ:

- حساب عدد عمليات Checkout الحالية
- السماح بعدد محدد فقط
- رفض الطلبات الزائدة مؤقتاً

الحد الأقصى

```
MAX_CONCURRENT = 5
```

إذا تجاوز النظام هذا العدد:
يرجع:

503 Service Unavailable

الفائدة

هذا يمنع:

- انهيار السيرفر
- استهلاك الذاكرة بشكل مفرط
- بطء الاستجابة

6. Requirement 3 — Asynchronous Processing (Queues)

الهدف

فصل العمليات الثقيلة عن الـ Request الرئيسي.

بدلاً من انتظار:

- إنشاء الفاتورة
- إرسال الإشعارات

يتم تنفيذها بالخلفية.

الحل المستخدم

تم استخدام:

- Laravel Queues
- Jobs

الوظائف المنفذة بالخلفية

[GenerateInvoiceJob](#)

توليد الفاتورة.

SendOrderNotificationJob

إرسال إشعار للمستخدم.

كيف يعمل النظام

بعد نجاح عملية الدفع:

```
GenerateInvoiceJob::dispatch(...)  
SendOrderNotificationJob::dispatch(...)
```

تُرسل الوظائف إلى Queue ،
ويتم تنفيذها بواسطة:

```
php artisan queue:work
```

الفائدة

- تقليل زمن استجابة الـ API
- تحسين تجربة المستخدم
- زيادة قدرة النظام على تحمل الضغط

7. Requirement 4 — Batch Processing

المشكلة

معالجة جميع الطلبات دفعة واحدة يستهلك:

- ذاكرة RAM كبيرة
- وقت أطول
- ضغط أعلى على السيرفر

الحل المستخدم

تم استخدام:

```
chunkById()
```

داخل:

```
DailySalesReportService
```

Before Optimization

->get()

تحميل جميع الطلبات مرة واحدة داخل الذاكرة.

After Optimization

->chunkById(1000)

معالجة البيانات على دفعات صغيرة.

الفوائد

- تقليل استهلاك الذاكرة
 - استقرار أفضل
 - دعم البيانات الكبيرة
 - تحسين الأداء تحت الضغط
-

8. Requirement 5 — Load Distribution

الهدف

محاكاة نظام موزع يحتوي على عدة Servers.

الحل المستخدم

تم تشغيل 3 Laravel Instances

Instance	Port
laravel-node-1	8001
laravel-node-2	8002

Instance Port

laravel-node-3 8003

Load Balancer

تم استخدام:
NGINX

لتوزيع الطلبات باستخدام:

Round Robin

كيف يعمل

يقوم NGINX بتحويل كل Request إلى Server مختلف بالتتابع:

8001 → 8002 → 8003 → 8001

10. Performance Testing

تم استخدام:
k6

لاختبار:

- Concurrent Requests
 - Batch Processing
 - Load Distribution
 - Server Performance
 - Race Condition Handling
 - Concurrency Control
 - Queue Processing
 - Resource Management
 - Response Time Monitoring
 - Response Time Monitoring
-

الاختبارات المنفذة

الاختبار	الوصف
batch-before.js	اختبار المعالجة التقليدية (Synchronous Processing) قبل استخدام Chunk Processing
batch-after.js	اختبار المعالجة باستخدام Chunk Processing بعد تحسين الأداء
load-distribution.js	اختبار توزيع الأحمال (Load Balancer Distribution) بين السيرفرات
before_protection.js	اختبار النظام قبل تطبيق الحماية (بدون Rate Limiting أو Concurrency Control)
after_protection.js	اختبار النظام بعد تطبيق الحماية وإدارة الموارد
before_queue.js	اختبار المعالجة المتزامنة (Synchronous Queue Handling) قبل استخدام الـ Queue
after_queue.js	اختبار المعالجة غير المتزامنة (Asynchronous Queue Processing) بعد استخدام الـ Queue
race-condition.js	اختبار الـ Race Condition والتحقق من سلامة التزامن ومنع تعارض العمليات
metrics-helper.js	ملف مساعد لتجميع وتحليل مقاييس الأداء (CPU / RAM / Response Time) وإنشاء التقارير الموحدة

AOP Performance Monitoring

تم تطبيق مفهوم

Aspect-Oriented Programming (AOP)

لمراقبة أداء النظام دون تعديل منطق الأعمال الأساسي. (Business Logic)

بدلاً من إضافة كود القياس داخل كل Controller ،
تم إنشاء Middleware مستقل باسم:

PerformanceMonitor

يقوم تلقائياً بقياس:

- وقت تنفيذ الطلب (Execution Time)
- استهلاك الذاكرة (Memory Usage)
- استهلاك المعالج (CPU Usage)

لكل Request داخل النظام.

Middleware Implementation

يعتمد الـ Middleware على:

- `microtime(true)` لحساب زمن التنفيذ
- `memory_get_usage()` لحساب استهلاك الذاكرة
- `getrusage()` لحساب استهلاك المعالج

ثم يتم إرسال النتائج داخل: HTTP Headers

Header	Description
X-Server-Time-Ms	زمن التنفيذ بالمللي ثانية
X-Server-Memory-Mb	استهلاك الذاكرة
X-Server-CPU-Usage	استهلاك المعالج

هذا الأسلوب يحقق مفهوم AOP لأن عملية المراقبة تمت بشكل منفصل عن منطق التطبيق الأساسي.